

Séance 3 – Étudiant



Séance 3 – Base MariaDB & connexion Node-RED

Objectifs de la séance :

- Créer une base MariaDB et une table `capteurs` adaptées à nos mesures.
- Comprendre le choix des types (`FLOAT`, `DATETIME`, `VARCHAR`).
- Connecter Node-RED à MariaDB.
- Tester une insertion de valeurs fixes depuis Node-RED.
- Vérifier la réception des données avec une requête `SELECT`.

0) Prérequis

- Raspberry Pi opérationnelle (séance 1) avec Docker et Docker Compose.
- Conteneurs MariaDB et Node-RED lancés (séance 1).
- Accès au terminal (SSH) et à l'éditeur Node-RED (<http://<IP>:1880>).

Astuce : si vous avez suivi le cours, le service de base de données s'appelle "mariadb" dans docker-compose.yml. Cela sera le "host" côté Node-RED.

1) Créer la base & l'utilisateur (via terminal)

Ouvrez un terminal sur la Raspberry Pi en ssh.

a) Entrer dans le conteneur MariaDB (méthode Docker)

```
docker exec -it mariadb mariadb -uroot -p
```

► Explication L1: Exécute le binaire client 'mariadb' à l'intérieur du conteneur nommé 'mariadb' (-it = interactif, -u root = utilisateur root, -p = demander le mot de passe).

Saisissez le mot de passe `root` configuré dans votre `docker-compose.yml` (variable `MARIADB_ROOT_PASSWORD`).

b) Créer la base, l'utilisateur et la table

Dans le client MariaDB, exécutez pas à pas (copier/coller autorisé) :

```
CREATE DATABASE IF NOT EXISTS iot_lab
```

```
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

```
CREATE USER IF NOT EXISTS 'iot_user'@'%' IDENTIFIED BY 'iot_pass';
```

```
GRANT ALL PRIVILEGES ON iot_lab.* TO 'iot_user'@'%';
```

```
FLUSH PRIVILEGES;
```

```
USE iot_lab;
```

```
CREATE TABLE IF NOT EXISTS capteurs (
```

```
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
```

```
device_id VARCHAR(64) NOT NULL,
```

```
capteur VARCHAR(32) NOT NULL,
```

```
valeur FLOAT NOT NULL,
```

```
unite VARCHAR(16) NOT NULL,
```

```
ts DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
INDEX idx_ts (ts),
```

```
INDEX idx_device_ts (device_id, ts)
```

```
) ENGINE=InnoDB;
```

- Explication L1: Crée la base nommée `iot_lab` si elle n'existe pas déjà.
- Explication L2: Force l'encodage UTF-8 moderne et un tri Unicode cohérent.
- Explication L3: Crée l'utilisateur `'iot_user'` (accessible depuis n'importe quelle IP `'%'`) avec le mot de passe `'iot_pass'`.

- Explication L4: Accorde tous les droits sur le schéma `iot_lab` à cet utilisateur (lecture/écriture/DDL).
- Explication L5: Rend les changements de privilèges effectifs immédiatement.
- Explication L6: Sélectionne la base `iot_lab` pour les instructions suivantes.
- Explication L7: Crée la table `capteurs` uniquement si elle n'existe pas déjà.
- Explication L8: Colonne identifiant unique non signé, incrémentée automatiquement (clé primaire).
- Explication L9: Identifiant de l'appareil/émetteur de la mesure (ex. 'arduino-01').
- Explication L10: Type de capteur (ex. 'DHT22', 'LDR', etc.).
- Explication L11: Valeur mesurée (réel simple précision).
- Explication L12: Unité de la mesure (ex. 'C', '%', 'lx').
- Explication L13: Horodatage de la mesure ; par défaut, prend l'instant de l'insertion.
- Explication L14: Index simple sur `ts` pour accélérer les requêtes par date.
- Explication L15: Index composite pour filtrer efficacement par `device` + période.
- Explication L16: Moteur transactionnel InnoDB (ACID, index, FK, etc.).

Qu'est-ce qu'un moteur de stockage ?

- MariaDB/MySQL peut gérer plusieurs "moteurs" pour stocker physiquement les données.
- Chaque moteur a ses caractéristiques (performance, fonctionnalités, contraintes).
- Les deux plus connus :
 - MyISAM (ancien, rapide en lecture, mais pas transactionnel, pas de clés étrangères).
 - InnoDB (moderne, par défaut aujourd'hui).

Pourquoi InnoDB ?

- Transactionnel → supporte les transactions **ACID** :
 - **Atomique** : une transaction réussit entièrement ou est annulée (pas de demi-état).
 - **Cohérente** : les contraintes (clés primaires, uniques, etc.) sont respectées.
 - **Isolée** : plusieurs transactions en parallèle ne se marchent pas dessus.
 - **Durable** : une fois validée (COMMIT), la donnée est conservée même en cas de crash.

- Clés étrangères (FK) → permet de définir des relations entre tables (capteurs ↔ devices par exemple) avec intégrité référentielle.
- Index efficaces → B-Tree, gestion des indexes multi-colonnes, optimisé pour les recherches.
- Verrouillage au niveau ligne (pas table entière) → meilleur en environnement multi-utilisateurs.
- Crash recovery → journalisation interne qui permet de retrouver un état cohérent après coupure.

c) Vérifier

```
SHOW DATABASES;
```

```
+-----+
| Database |
+-----+
| information_schema |
| iot_lab |
| iotdb |
| mysql |
| performance_schema |
| sys |
+-----+
```

```
USE iot_lab;
```

```
Database changed
MariaDB [iot_lab]> use iotdb
Database changed
MariaDB [iotdb]> USE iot_lab;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [iot_lab]> █
```

```
SHOW TABLES;
```

```
Database changed
MariaDB [iot_lab]> SHOW TABLES;
```

```
+-----+
| Tables_in_iot_lab |
+-----+
| capteurs           |
+-----+
```

```
MariaDB [iot_lab]> DESCRIBE capteurs;
```

```
+-----+-----+-----+-----+-----+-----+
| Field      | Type                | Null | Key | Default        | Extra          |
+-----+-----+-----+-----+-----+-----+
| id         | bigint(20) unsigned | NO   | PRI | NULL           | auto_increment |
| device_id  | varchar(64)         | NO   | MUL | NULL           |                |
| capteur    | varchar(32)         | NO   |     | NULL           |                |
| valeur     | float               | NO   |     | NULL           |                |
| unite      | varchar(16)         | NO   |     | NULL           |                |
| ts         | datetime            | NO   | MUL | current_timestamp() |                |
+-----+-----+-----+-----+-----+-----+
```

```
DESCRIBE capteurs;
```

```
Database changed
MariaDB [iot_lab]> describe capteurs;
```

```
+-----+-----+-----+-----+-----+-----+
| Field      | Type                | Null | Key | Default        | Extra          |
+-----+-----+-----+-----+-----+-----+
| id         | bigint(20) unsigned | NO   | PRI | NULL           | auto_incr
ement |
| device_id  | varchar(64)         | NO   | MUL | NULL           |                |
| capteur    | varchar(32)         | NO   |     | NULL           |                |
| valeur     | float               | NO   |     | NULL           |                |
| unite      | varchar(16)         | NO   |     | NULL           |                |
| ts         | datetime            | NO   | MUL | current_timestamp() |                |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.007 sec)
```

Pourquoi ces types ?

- 'VARCHAR' pour les identifiants (souples et courts) ;
 - Ce que c'est : chaîne de longueur variable (stockage = longueur réelle en octets + 1 ou 2 octets d'en-tête).
 - Encodage : avec utf8mb4, un caractère = 1 à 4 octets. Une colonne VARCHAR(64) peut donc coûter jusqu'à 256 octets + en-tête.
 - Tri/égalité : dépend de la collation.

- utf8mb4_unicode_ci ⇒ insensible à la casse ("ABC" = "abc").
 - Pour des identifiants stricts (device_id) où la casse compte, préférer utf8mb4_bin ou VARBINARY.
 - Indexation : indexable, mais attention aux très grandes longueurs (on utilise parfois un prefix index, ex. INDEX (device_id(16))).
 - Quand l'utiliser : identifiants, noms, codes capteur.
 - Éviter : pour des longueurs fixes (ex. codes ISO à 2 lettres), CHAR(2) est plus adapté.
-
- `FLOAT` pour les mesures numériques non entières ;
 - Ce que c'est : nombre à virgule flottante en simple précision (~7 chiffres significatifs).
Stockage ≈ 4 octets. Il existe aussi DOUBLE (~15-16 chiffres, 8 octets).
 - Avantages : rapide, peu gourmand, parfait pour mesures physiques (température, luminosité, etc.) où une petite erreur d'arrondi n'est pas critique.
 - Limites : approximatif par nature (représentation binaire), peut afficher des 23.399999 au lieu de 23.4.
Pour des valeurs financières ou des calculs où l'exactitude décimale est cruciale, utiliser DECIMAL(p,s) (stockage exact en décimal).
 - Quand l'utiliser : télémétrie IoT "classique" (°C, %, Pa, m/s, etc.), graphiques, seuils.
 - Alternatives :
 - DOUBLE si l'on souhaite accumuler ou dériver des valeurs et limiter la propagation d'erreurs.
 - DECIMAL(10,3) si tu veux des valeurs décimales exactes.
-
- `DATETIME` pour un horodatage lisible côté SQL (pratique pour requêtes par date) ;
 - Ce que c'est : date+heure sans fuseau horaire (format YYYY-MM-DD HH:MM:SS).
 - Point clé : aucune conversion de fuseau. La valeur stockée = la valeur fournie.
C'est simple pour des requêtes par période, mais c'est à l'utilisateur de décider s'il stocke en UTC ou en heure locale.
 - Fraction de seconde : DATETIME(3) pour les millisecondes, DATETIME(6) pour les microsecondes.
 - Indexation : très bon critère d'index pour les requêtes temporelles et l'archivage (ex. INDEX(ts)).
 - Par rapport à TIMESTAMP : TIMESTAMP stocke en UTC et convertit selon le fuseau de session (pratique, mais plus "magique" et plafonné vers 2038 sur certaines versions). DATETIME est neutre, donc prévisible.

Bonnes pratiques IoT

- Choisir une convention :
 - UTC partout (recommandé en multi-sites) et convertir côté affichage, ou
 - Local si tout est sur un site et que l'on souhaite stocker et lire les heures dans le fuseau horaire local.

- Index sur `(device\_id, ts)` pour accélérer les filtres « par capteur et période ».

2) Installer le nœud MySQL dans Node-RED (si nécessaire)

1. Ouvrez Node-RED → menu ☰ → Manage palette → Install.
2. Recherchez `node-red-node-mysql` et installez-le.
3. Redéployez Node-RED si demandé.

Ce nœud permet d'envoyer des requêtes SQL depuis Node-RED. La requête est mise dans `msg.topic`.

3) Configurer la connexion MariaDB dans Node-RED

4. Faites glisser un nœud `mysql` dans l'éditeur, double-cliquez-le.
5. À droite de Server, cliquez sur le crayon ✎ pour créer une nouvelle connexion :
 - Host : `mariadb` *(nom du service Docker, recommandé)* ou `127.0.0.1` si Node-RED et MariaDB tournent hors Docker sur la même machine.
 - Port : `3306`
 - Database : `iot\_lab`
 - User : `iot\_user`
 - Password : `iot\_pass`
 - Cochez Use connection pool.
6. Enregistrez (Add) puis Done.

Si vous recevez `ECONNREFUSED` : vérifiez host/port et que le conteneur MariaDB tourne.

4) Tester une insertion “de valeurs fixes”

A) Préparer un objet JSON de test

Ajoutez un nœud Inject configuré en JSON avec ce contenu :

```
{ "device_id": "arduino-01", "capteur": "DHT22", "valeur": 23.4,
  "unite": "C", "ts": "auto" }
```

B) Convertir en requête SQL « préparée »

Ajoutez un nœud Function nommé To SQL (INSERT) avec :

```
const p = msg.payload;

const ts = (p.ts === "auto" || !p.ts) ? new Date() : new Date(p.ts);

// format "YYYY-MM-DD HH:MM:SS"

const pad = n => String(n).padStart(2, '0');

const toSQL = d => d.getFullYear()+"-"+pad(d.getMonth()+1)+"-
"+pad(d.getDate())+
"
"+pad(d.getHours())+": "+pad(d.getMinutes())+": "+pad(d.getSeconds());

msg.topic = "INSERT INTO capteurs (device_id, capteur, valeur, unite,
ts) VALUES (?, ?, ?, ?, ?)";

msg.payload = [ p.device_id, p.capteur, Number(p.valeur), p.unite,
toSQL(ts) ];

return msg;
```

- Explication L1: Récupère la charge utile (objet JSON issu de l'Inject).
- Explication L2: Si 'ts' vaut 'auto' ou est absent, on utilise l'heure courante ; sinon on parse la valeur transmise.
- Explication L3: Commentaire : on va formater la date dans un format SQL lisible.
- Explication L4: Petite fonction utilitaire : complète à 2 chiffres (01,02,...).
- Explication L5: Construit une chaîne 'YYYY-MM-DD' +
- Explication L6: ' HH:MM:SS' pour un DATETIME SQL standard.
- Explication L7: Définit la requête SQL paramétrée : 5 paramètres correspondent aux 5 '?'.

► Explication L8: Construit le tableau de paramètres dans le bon ordre ; coercition numérique sur 'valeur'.

► Explication L10: Renvoie le message au nœud MySQL.

C) Envoyer vers le nœud MySQL

Reliez Inject → Function → MySQL → Debug (debug sur complete msg).

Cliquez sur Inject : vous devez obtenir `affectedRows: 1`.

Pourquoi requêtes préparées ? Pour éviter les risques d'injection SQL et gérer proprement les types.

5) Vérifier côté base (dans Node-RED ou SQL)

Option 1 – depuis Node-RED

Créez un deuxième Inject (string) → Function (SELECT) → MySQL → Debug :

```
msg.topic = "SELECT id, device_id, capteur, valeur, unite, ts FROM  
capteurs ORDER BY id DESC LIMIT 5";
```

```
return msg;
```

Vous verrez un tableau de lignes dans le debug.

Option 2 – depuis le terminal SQL

```
USE iot_lab;
```

```
SELECT COUNT(*) AS n FROM capteurs;
```

```
SELECT * FROM capteurs ORDER BY id DESC LIMIT 5;
```

6) Dépannage rapide

- `ECONNREFUSED` ⇒ host/port/nom du service Docker incorrects, ou base arrêtée.
- `ER\_ACCESS\_DENIED\_ERROR` ⇒ mauvais utilisateur/mot de passe, ou privilèges non accordés.

- Insertion sans erreur mais valeurs nulles $\Rightarrow$ vérifiez le type de ``msg.payload`` (objet/array) et le nombre d'interrogations ``?`` qui doit correspondre au nombre de valeurs.
- Heures décalées $\Rightarrow$ vérifier le fuseau (``TZ`` du conteneur) ou stocker un UTC cohérent et convertir côté affichage.